

Smart Sleep Environment Analyzer

MEMORIA DEL PROYECTO

Asignatura: Procesadores Digitales

Curso: 2025-2026

Autores: Martina Luna y Gerard Bustillo

Fecha: Junio 2026

ÍNDICE

1. INTRODUCCIÓN Y OBJETIVOS	2
1.1. Contexto	2
1.2. Propósito del proyecto	2
1.3. Objetivos específicos	3
2. IDEA PRINCIPAL DEL PROYECTO	3
3. FUNCIONALIDADES DEL SISTEMA	4
3.1. Sleep Score (Puntuación de sueño)	5
3.2. Best Hour (Mejor franja horaria)	7
3.3. Sleep Timeline (Línea temporal del sueño)	8
4. HARDWARE UTILIZADO	9
5. ESQUEMA DE CONEXIONES (PINOUT)	11
6. ESTRUCTURA DEL CÓDIGO	12
6.1. Árbol de directorios	12
6.2. Principios de diseño aplicados	13
7. ARQUITECTURA DEL SOFTWARE	14
8. TAREAS FREERTOS	15
9. MECANISMOS DE SINCRONIZACIÓN	16
9.1. Colas (Queues)	16
9.2. Mutex	16
9.3. SessionManager	17
10. CONECTIVIDAD: STA + AP	17
11. INTERFAZ WEB	18
11.1. Pantalla principal	18
11.2. Ranking de sesiones	18
11.3. Historial de sesiones	18
11.4. Detalle de sesión	18
12. CONTROL DE CALIDAD Y PRUEBAS	19
12.1. Tests lógicos	19
12.2. Tests de hardware	19
12.3. Validación con referencias externas	20
13. MEJORAS IMPLEMENTADAS	20
13.1. OTA (Over-The-Air)	20
13.2. NTP (Network Time Protocol)	21
13.3. Robustez de la tarjeta SD	21
14. CONCLUSIONES	21
15. ANEXOS	23
15.1. Fotografías del hardware	23
15.2. Capturas de la interfaz web	24
15.3. Video demostrativo	25

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Contexto

En los últimos años, la calidad del sueño se ha convertido en un tema de gran relevancia debido a su impacto directo en la salud física y mental de las personas. Factores ambientales como la calidad del aire, la temperatura, la humedad o la iluminación pueden influir significativamente en el descanso nocturno. Sin embargo, la mayoría de las personas no tienen acceso a herramientas que les permitan medir y analizar estas variables de forma objetiva y continua.

El presente proyecto nace con la intención de ofrecer una solución accesible, de bajo coste y fácil de usar que permita a cualquier persona monitorizar las condiciones de su dormitorio durante la noche y recibir información útil para mejorar su descanso.

1.2. Propósito del proyecto

El proyecto consiste en el desarrollo de un analizador del entorno de sueño basado en un microcontrolador ESP32. El sistema es capaz de monitorizar en tiempo real las condiciones ambientales de una habitación mientras el usuario duerme, evaluando parámetros clave como la concentración de CO₂, la temperatura, la humedad relativa y el nivel de iluminación.

1.3. Objetivos específicos

Los objetivos que nos planteamos al inicio del desarrollo fueron los siguientes:

- Monitorizar en tiempo real los parámetros ambientales (CO₂, temperatura, humedad y luz) durante toda la noche.
- Evaluar la calidad del entorno mediante un Sleep Score, una puntuación de 0 a 100 que refleja lo favorable que ha sido el ambiente para el descanso.
- Identificar la mejor franja horaria de la noche, es decir, el período de una hora en el que las condiciones han sido más óptimas.
- Generar alertas y recomendaciones personalizadas cuando algún parámetro se salga de los rangos considerados saludables.
- Almacenar los datos en una tarjeta microSD para su posterior análisis y consulta.
- Visualizar los datos a través de una interfaz web accesible desde cualquier dispositivo conectado a la red del dispositivo.
- Permitir la actualización inalámbrica del firmware (OTA) para poder mejorar el sistema sin necesidad de acceder físicamente al ESP32.

2. IDEA PRINCIPAL DEL PROYECTO

La idea principal del proyecto es desarrollar un sistema inteligente capaz de analizar el entorno de sueño en tiempo real y determinar cómo las condiciones ambientales , como la calidad del aire, la temperatura, la humedad o la luz, afectan directamente al descanso del usuario.

El objetivo final es proporcionar recomendaciones personalizadas que ayuden a mejorar la calidad del sueño y, por tanto, el bienestar general de la persona. Todo ello con un enfoque de bajo coste, fácil de usar y accesible desde cualquier dispositivo con navegador.

3. FUNCIONALIDADES DEL SISTEMA

A continuación se detallan las funcionalidades que ofrece el sistema una vez puesto en marcha:

Funcionalidad	Descripción
<u>Medición de parámetros</u>	El sistema mide continuamente la concentración de CO ₂ (ppm), la temperatura (°C), la humedad relativa (%) y el nivel de luz (lux).
<u>Clasificación ambiental</u>	Cada parámetro se clasifica como bueno, aceptable o malo según los umbrales definidos en el archivo de configuración.
<u>Sleep Score</u>	Al finalizar la sesión, se calcula una puntuación global de 0 a 100 que refleja la calidad del entorno durante el sueño.
<u>Mejor franja horaria (Best Hour)</u>	El sistema analiza todos los datos y encuentra la hora de la noche en la que las condiciones fueron más favorables.
<u>Sleep Timeline</u>	Representación gráfica de la evolución de los parámetros a lo largo de la sesión, con zoom interactivo y visualización por variables.

<u>Alertas y recomendaciones</u>	Durante la sesión, si algún parámetro entra en rango crítico, se activa una alerta visual (LEDs) y sonora (buzzer), y se guarda una recomendación para el usuario.
<u>Almacenamiento en SD</u>	Todas las sesiones se guardan en la tarjeta microSD en dos formatos: CSV (datos brutos) y JSON (estadísticas y resultados).
<u>Interfaz web</u>	El sistema levanta un servidor web que permite visualizar los datos en tiempo real, consultar el historial de sesiones, ver el Sleep Score y las gráficas.
<u>Actualización OTA</u>	El firmware se puede actualizar de forma inalámbrica a través de la red WiFi del ESP32, sin necesidad de conectar el cable USB.

3.1. Sleep Score (Puntuación de sueño)

El Sleep Score es una puntuación de 0 a 100 que evalúa la calidad del entorno de sueño. Cuanto más alto es el número, mejores han sido las condiciones ambientales durante la sesión.

¿Cómo se calcula?

El Sleep Score se calcula sumando las puntuaciones parciales de cada variable ambiental, donde cada una tiene un peso diferente según su importancia en la calidad del sueño:

Variable	Puntuación máxima	Criterios de puntuación
<u>CO₂</u>	40 puntos	Menos de 800 ppm → 40 puntos / 800-1000 ppm → 32 puntos / 1000-1400 ppm → 20 puntos / 1400-1800 ppm → 10 puntos / Más de 1800 ppm → 0 puntos
<u>Temperatura</u>	25 puntos	18-22°C → 25 puntos / 22-24°C → 18 puntos / 24-26°C → 10 puntos / Menos de 18°C o más de 26°C → 0 puntos
<u>Humedad</u>	20 puntos	40-60% → 20 puntos / 30-40% o 60-70% → 12 puntos / Menos de 30% o más de 70% → 0 puntos
<u>Luz</u>	15 puntos	Menos de 5 lux → 15 puntos / 5-20 lux → 8 puntos / Más de 20 lux → 0 puntos

Fórmula: Sleep Score = CO₂_score + Temp_score + Hum_score + Light_score

Interpretación del Sleep Score:

Puntuación	Interpretación	Significado
85 – 100	Condiciones óptimas	El entorno ha sido ideal para el sueño
70 – 84	Buenas condiciones	El entorno ha sido favorable
50 – 69	Condiciones aceptables	El entorno ha sido mejorable
30 – 49	Condiciones desfavorables	El entorno ha sido poco adecuado
0 – 29	Condiciones críticas	El entorno ha sido perjudicial para el sueño

3.2. Best Hour (Mejor franja horaria)

La Best Hour es el período de una hora dentro de la sesión en el que las condiciones ambientales han sido más favorables para el sueño. El sistema analiza todos los datos de la sesión y encuentra el intervalo de 60 minutos con la mejor puntuación global.

¿Cómo se calcula?

El algoritmo de búsqueda funciona de la siguiente manera:

1. **Ventana deslizante:** El sistema utiliza una ventana de 120 muestras (1 hora, ya que las lecturas son cada 30 segundos) que se desplaza a lo largo de todos los datos de la sesión.
2. **Puntuación de cada ventana:** Para cada ventana, se calcula una puntuación de penalización basada en los valores de CO₂, temperatura, humedad y luz. La ventana con la menor penalización se selecciona como la mejor franja horaria.

3. **Conversión a hora real:** El tiempo relativo de la ventana se convierte a hora real usando el timestamp de inicio de sesión guardado al principio.

3.3. Sleep Timeline (Línea temporal del sueño)

El Sleep Timeline es la representación gráfica de la evolución de los parámetros ambientales a lo largo de la sesión. Permite al usuario visualizar cómo han variado el CO₂, la temperatura, la humedad y la luz durante toda la noche.

¿Cómo se genera?

Durante la sesión, StorageTask guarda cada lectura en un archivo CSV. Al finalizar, AnalysisTask procesa los datos y la página web utiliza la librería Chart.js para dibujar las gráficas. El ESP32 envía los datos en formato JSON a través de la API, y el navegador los representa gráficamente.

Funcionalidades interactivas:

Funcionalidad	Descripción
<u>Zoom</u>	El usuario puede hacer zoom en períodos concretos (1h, 2h, 4h, 8h o Todo)
<u>Selección de variables</u>	El usuario puede elegir qué variable visualizar (CO ₂ , Temp, Hum, Luz)
<u>Tooltips</u>	Al pasar el cursor sobre un punto, se muestra el valor exacto y la hora
<u>Hora real</u>	El eje X muestra horas reales gracias al NTP

¿Por qué Chart.js en el cliente?

- El ESP32 tiene recursos limitados (328 KB de DRAM). Generar imágenes en el ESP32 consumiría demasiada memoria y CPU.
- Chart.js permite zoom, tooltips y selección de variables sin recargar la página.
- Chart.js está alojado en la tarjeta SD, no en internet, porque el ESP32 funciona en modo AP sin acceso a internet.

4. HARDWARE UTILIZADO

La elección de los componentes hardware se ha realizado teniendo en cuenta tres criterios principales: disponibilidad, facilidad de conexión y compatibilidad con el ESP32.

Componente	Modelo	Función	Bus de comunicación	Motivo de la elección
<u>Microcontr</u> <u>olador</u>	ESP32 DevKit V1	Procesador principal	-	Dos núcleos a 240 MHz, WiFi integrado, gran cantidad de pines GPIO
<u>Sensor de</u> <u>CO₂</u>	Sensirion SCD41	CO ₂ , temperatura y humedad	I ² C	Alta precisión, compensación integrada, comunicación I ² C
<u>Sensor de</u> <u>luz</u>	BH1750	Iluminación en lux	I ² C	Bajo coste, alta precisión, lectura directa en lux

<u>Pantalla</u> <u>OLED</u>	SH1106 128x64	Visualización de datos	I ² C	Bajo consumo, alto contraste, resolución suficiente
<u>Tarjeta SD</u>	Módulo microSD	Almacenamiento de datos	SPI	Almacenamiento persistente de grandes volúmenes de datos
<u>LED RGB</u>	3 LEDs individuales	Indicadores de estado	GPIO	Retroalimentación visual inmediata
<u>Buzzer</u>	Buzzer pasivo	Alarma sonora	GPIO	Permite generar diferentes frecuencias y melodías
<u>Botón</u>	Pulsador	Control de sesión	GPIO	Inicio y fin de sesión sin necesidad de la web

5. ESQUEMA DE CONEXIONES (PINOUT)

Componente	Pines	Bus	Observaciones
<u>SCD41 (CO₂)</u>	SDA: GPIO21, SCL: GPIO22	I ² C	Pines estándar del bus I ² C hardware
<u>BH1750 (Luz)</u>	SDA: GPIO21, SCL: GPIO22	I ² C	Comparte el mismo bus que SCD41 y OLED
<u>OLED SH1106</u>	SDA: GPIO21, SCL: GPIO22	I ² C	Dirección 0x3C por defecto
<u>MicroSD</u>	CS: GPIO5, SCK: GPIO18, MOSI: GPIO23, MISO: GPIO19	SPI	Velocidad reducida a 4 MHz para estabilidad
<u>LED Rojo</u>	GPIO13	GPIO	Salida digital activa a nivel alto
<u>LED Amarillo</u>	GPIO14	GPIO	Salida digital activa a nivel alto
<u>LED Verde</u>	GPIO27	GPIO	Salida digital activa a nivel alto
<u>Buzzer</u>	GPIO26	GPIO	Salida PWM para generar tonos
<u>Botón</u>	GPIO25	GPIO	Entrada digital con pull-up interna, activo a LOW

6. ESTRUCTURA DEL CÓDIGO

La organización del código sigue una estructura basada en las convenciones de PlatformIO y en las buenas prácticas de ingeniería de software embebido.

6.1. Árbol de directorios

text

```
Smart_Sleep_Environment_Analyzer_PROYECTO/
├── platformio.ini      # Configuración del entorno
├── include/
│   └── config.h        # Definiciones globales
├── lib/
│   └── drivers/        # Drivers de sensores
│       ├── BH1750.cpp/h
│       └── SCD41.cpp/h
├── src/
│   ├── main.cpp        # Punto de entrada
│   ├── SessionManager.cpp/h  # Gestión de sesiones
│   ├── network/        # Módulos de red
│   │   ├── NTPManager.cpp/h
│   │   └── OTAManager.cpp/h
│   ├── tasks/          # Tareas FreeRTOS
│   │   ├── task_Sensor.cpp/h
│   │   ├── task_Display.cpp/h
│   │   ├── task_Alert.cpp/h
│   │   ├── task_Storage.cpp/h
│   │   ├── task_Analysis.cpp/h
│   │   ├── task_Button.cpp/h
│   │   └── task_WebServer.cpp/h
│   └── test/           # Tests
└── docs/              # Documentación
```

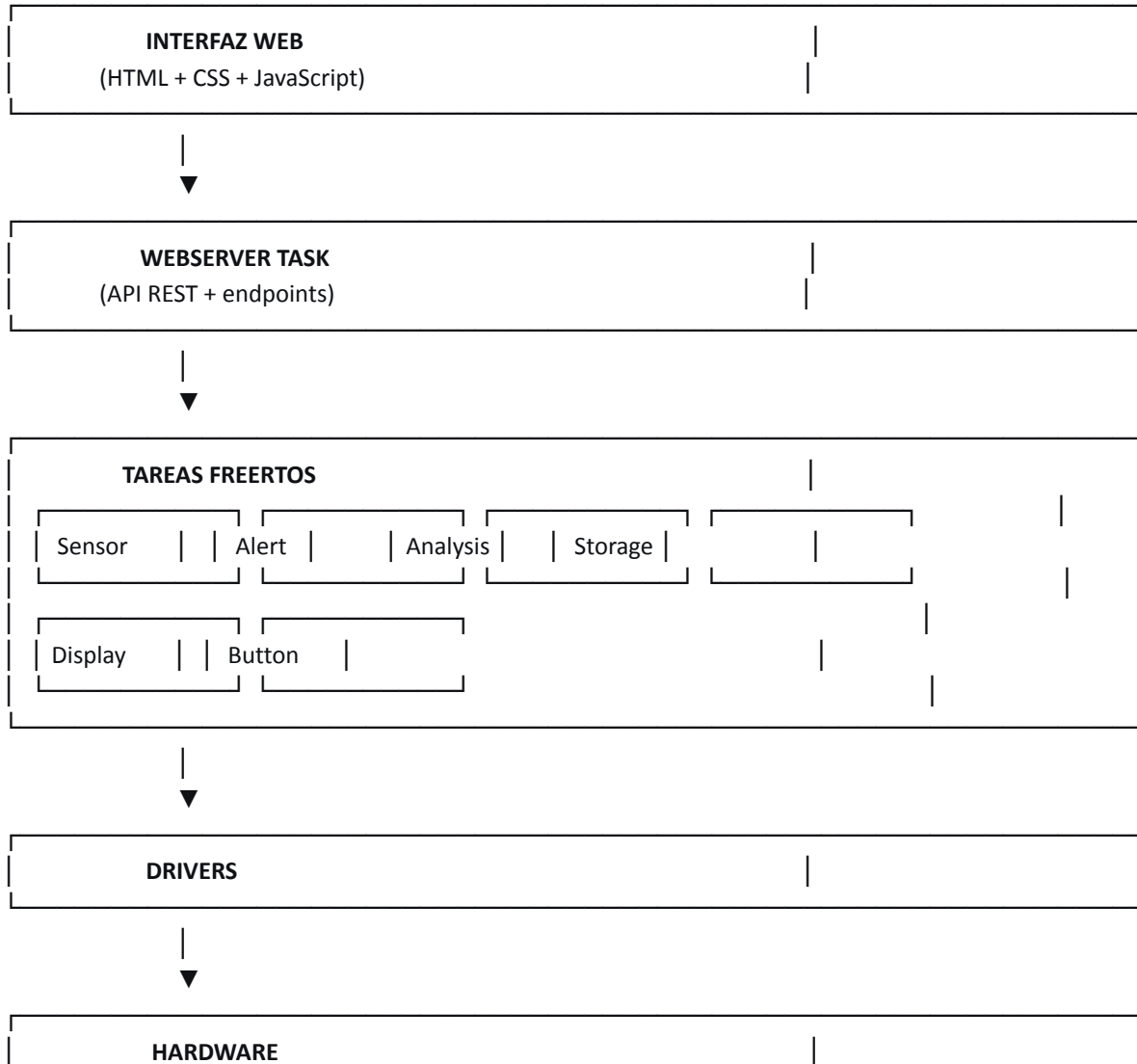
6.2. Principios de diseño aplicados

Principio	Aplicación en el proyecto
<u>Modularidad</u>	Cada funcionalidad está encapsulada en un módulo independiente
<u>Encapsulación</u>	Los drivers ocultan la complejidad de la comunicación con los sensores
<u>Centralización</u>	La configuración está centralizada en config.h
<u>Concurrencia</u>	FreeRTOS permite ejecutar múltiples tareas en paralelo
<u>Comunicación mediante colas</u>	Las tareas se comunican mediante colas, evitando variables globales
<u>Sincronización con mutex</u>	Los recursos compartidos están protegidos con mutex

7. ARQUITECTURA DEL SOFTWARE

La arquitectura sigue un modelo de capas bien diferenciadas:

text



Flujo de datos:

1. SensorTask lee los sensores cada 30 segundos y envía los datos a DisplayTask, AlertTask y StorageTask.
2. DisplayTask actualiza la pantalla OLED.
3. AlertTask evalúa los datos y activa LEDs y buzzer si es necesario.

4. StorageTask escribe los datos en la SD.
5. SessionManager notifica los cambios de sesión a todas las tareas.
6. AnalysisTask procesa los datos al finalizar la sesión y genera el Sleep Score.
7. WebServerTask expone los datos a través de la API REST.

8. TAREAS FREERTOS

Tarea	Núcleo	Prioridad	Función	Periodicidad
<u>SensorTask</u>	Núcleo 0	4 (Alta)	Lectura de sensores	30 segundos
<u>AlertTask</u>	Núcleo 1	4 (Alta)	Alertas, LEDs y buzzer	En cada lectura
<u>WebServerTask</u>	Núcleo 1	2 (Media)	Servidor web y API	Continuo
<u>AnalysisTask</u>	Núcleo 1	2 (Media)	Sleep Score y estadísticas	Al finalizar sesión
<u>DisplayTask</u>	Núcleo 1	2 (Media)	Pantalla OLED	1 segundo
<u>ButtonTask</u>	Núcleo 1	2 (Media)	Botón con antirrebote	10 ms
<u>StorageTask</u>	Núcleo 0	1 (Baja)	Escritura en SD	En cada lectura

9. MECANISMOS DE SINCRONIZACIÓN

9.1. Colas (Queues)

Cola	Tamaño	Tipo	Función
<u>sensorQueueForDisplay</u>	10	SensorData	Sensor → Display
<u>sensorQueueForAlert</u>	10	SensorData	Sensor → Alert
<u>sensorQueueForStorage</u>	10	SensorData	Sensor → Storage
<u>sessionQueue*</u>	5	SessionCommand	SessionManager → Tareas
<u>recommendationQueue</u>	5	Recommendation	Alert → Display

9.2. Mutex

Mutex	Protege	Uso
<u>_dataMutex</u>	_currentData en WebServerTask	Protege el acceso a los datos actuales
<u>_mutex</u>	_sessionActive en SessionManager	Protege el estado de la sesión

9.3. SessionManager

El SessionManager coordina el estado de la sesión. Cuando se inicia o finaliza una sesión, notifica a todas las tareas suscritas mediante colas de comandos.

10. CONECTIVIDAD: STA + AP

El ESP32 funciona en modo STA+AP (Estación + Punto de Acceso) por dos razones:

AP (Access Point) — Obligatorio para la interfaz web

El ESP32 crea su propia red WiFi con SSID SmartSleep_Analyzer. Esto permite que cualquier dispositivo se conecte directamente al ESP32 sin necesidad de un router WiFi externo, garantizando el acceso a la página web siempre.

STA (Station) — Necesario para el NTP y la hora real

El modo STA permite que el ESP32 se conecte a un router WiFi externo y obtenga la hora real desde servidores NTP. Esto es fundamental para que el Sleep Score, la Best Hour y el Sleep Timeline se muestren con marcas de tiempo reales.

Implementación:

cpp

```
void WebServerTask::setupAccessPoint() {  
  WiFi.mode(WIFI_AP_STA);  
  WiFi.softAP(AP_SSID, AP_PASSWORD, AP_CHANNEL, AP_HIDDEN);  
}
```

11. INTERFAZ WEB

La interfaz web permite al usuario interactuar con el sistema desde cualquier dispositivo con navegador.

11.1. Pantalla principal

- Datos en tiempo real: CO₂, temperatura, humedad y luz actualizados cada 3 segundos.
- Estado ambiental: Clasificación del ambiente en "Óptimo", "Aceptable" o "Desfavorable".
- Botones de control: Iniciar y finalizar sesión.
- Hora real gracias al NTP.

11.2. Ranking de sesiones

Lista de todas las sesiones ordenadas por Sleep Score (de mayor a menor), mostrando puntuación y duración.

11.3. Historial de sesiones

Lista de todas las sesiones ordenadas por fecha, permitiendo acceder al detalle de cada una.

11.4. Detalle de sesión

- Información general: Fecha, hora de inicio, hora de fin, duración y Sleep Score.
- Estadísticas: Medias, máximos y mínimos de cada variable.
- Sleep Timeline: Gráficas interactivas con zoom y selección de variables.
- Mejor franja horaria: Período de una hora con las mejores condiciones.
- Alertas y recomendaciones: Lista de eventos críticos y sugerencias de mejora.

12. CONTROL DE CALIDAD Y PRUEBAS

12.1. Tests lógicos

Test	Verifica
<u>test_averages.cpp</u>	Cálculo de medias y Sleep Score
<u>test_best_hours.cpp</u>	Cálculo de la mejor franja horaria
<u>test_quality_classifier.cpp</u>	Clasificación ambiental
<u>test_session_management.cpp</u>	Gestión de sesiones
<u>test_thresholds.cpp</u>	Coherencia de umbrales

12.2. Tests de hardware

Test	Verifica
<u>test_BH1750</u>	Sensor de luz
<u>test_button</u>	Botón físico
<u>test_buzzer</u>	Buzzer pasivo
<u>test_LEDs</u>	LEDs rojo, amarillo y verde
<u>test_microSD</u>	Tarjeta SD
<u>test_OLED_SH1106</u>	Pantalla OLED

12.3. Validación con referencias externas

Sensor	Método de validación
<u>Temperatura</u>	Comparación con el termostato de la habitación
<u>Humedad</u>	Comparación con la aplicación del tiempo del móvil
<u>Luz</u>	Tapar y destapar el sensor con la mano
<u>CO₂</u>	Soplar cerca del sensor y observar cómo sube y baja

13. MEJORAS IMPLEMENTADAS

13.1. OTA (Over-The-Air)

El sistema permite la actualización inalámbrica del firmware. Se ha desactivado mDNS para evitar conflictos con el modo WIFI_AP_STA.

Comando de actualización:

```
bash  
  
pio run --target upload --upload-port 192.168.4.1 -e ota
```

13.2. NTP (Network Time Protocol)

El sistema sincroniza la hora con servidores NTP para mostrar todas las marcas de tiempo en hora real. Si el WiFi se pierde durante la sesión, se utiliza el timestamp guardado al inicio.

13.3. Robustez de la tarjeta SD

- Múltiples intentos de inicialización (hasta 10)
- Espera progresiva entre intentos
- Reducción de la velocidad SPI a 4 MHz
- Reintentos de escritura y creación de archivos
- Reinicio del bus SPI en caso de fallo

14. CONCLUSIONES

El desarrollo del Smart Sleep Environment Analyzer ha demostrado ser una solución viable y funcional para monitorizar y analizar las condiciones ambientales durante el sueño. A lo largo del proyecto, hemos conseguido implementar un sistema completo que integra sensores de CO₂, temperatura, humedad y luz, una pantalla OLED para feedback local, una tarjeta SD para el almacenamiento persistente de los datos y una interfaz web accesible desde cualquier dispositivo.

Uno de los aspectos más destacados del proyecto es la capacidad del sistema para generar un Sleep Score objetivo y fiable, que permite al usuario conocer de forma cuantitativa la calidad de su entorno de sueño. Además, la identificación automática de la mejor franja horaria y la representación gráfica de la evolución de los parámetros a través del Sleep Timeline proporcionan una visión completa y útil para entender cómo mejorar el descanso.

Desde el punto de vista técnico, la arquitectura basada en FreeRTOS ha demostrado ser robusta y eficiente. La división del sistema en siete tareas concurrentes, cada una con su prioridad asignada, ha permitido que el sistema funcione de forma fluida y sin interferencias entre las diferentes funcionalidades. Los mecanismos de sincronización mediante colas y mutex han garantizado que los datos fluyan correctamente entre las tareas y que los recursos compartidos estén protegidos.

La implementación de la conectividad STA+AP ha sido clave para ofrecer una experiencia de usuario completa. El modo AP garantiza que la interfaz web esté siempre accesible, mientras que el modo STA permite obtener la hora real a través de NTP, lo que da sentido a funcionalidades como la Best Hour y el Sleep Timeline.

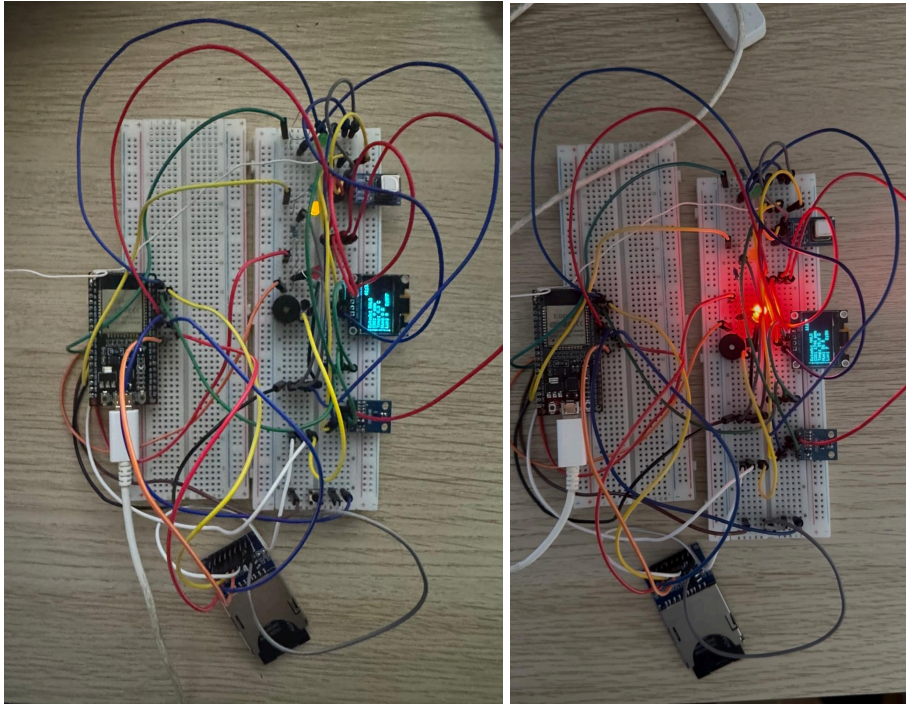
El sistema de actualización OTA ha resultado ser una mejora muy valiosa, ya que permite corregir errores o añadir nuevas funcionalidades sin necesidad de acceder físicamente al dispositivo. Esto facilita el mantenimiento y la evolución del sistema a largo plazo.

Por último, el control de calidad realizado mediante tests lógicos, pruebas de hardware y validación con referencias externas nos ha permitido confirmar que el sistema es fiable y que los sensores proporcionan valores coherentes con la realidad.

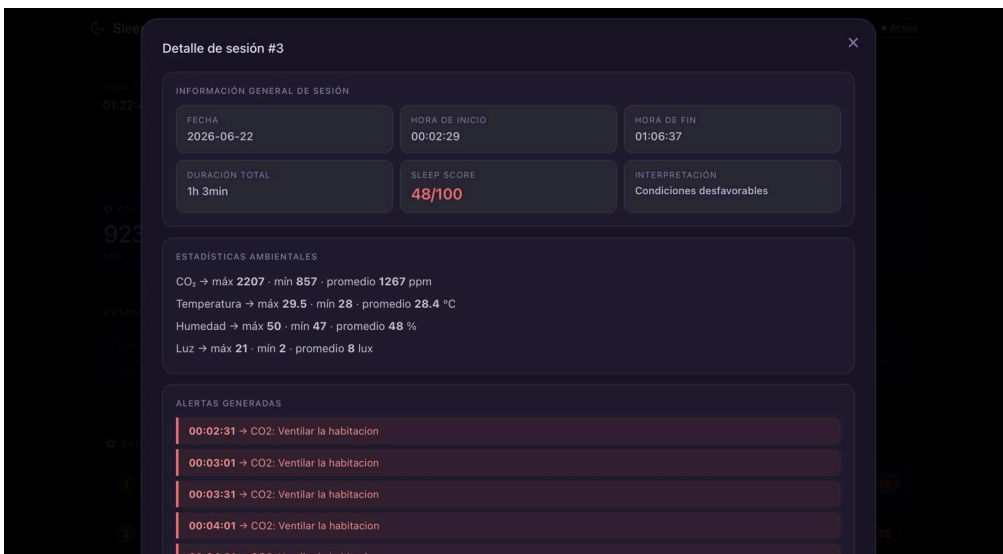
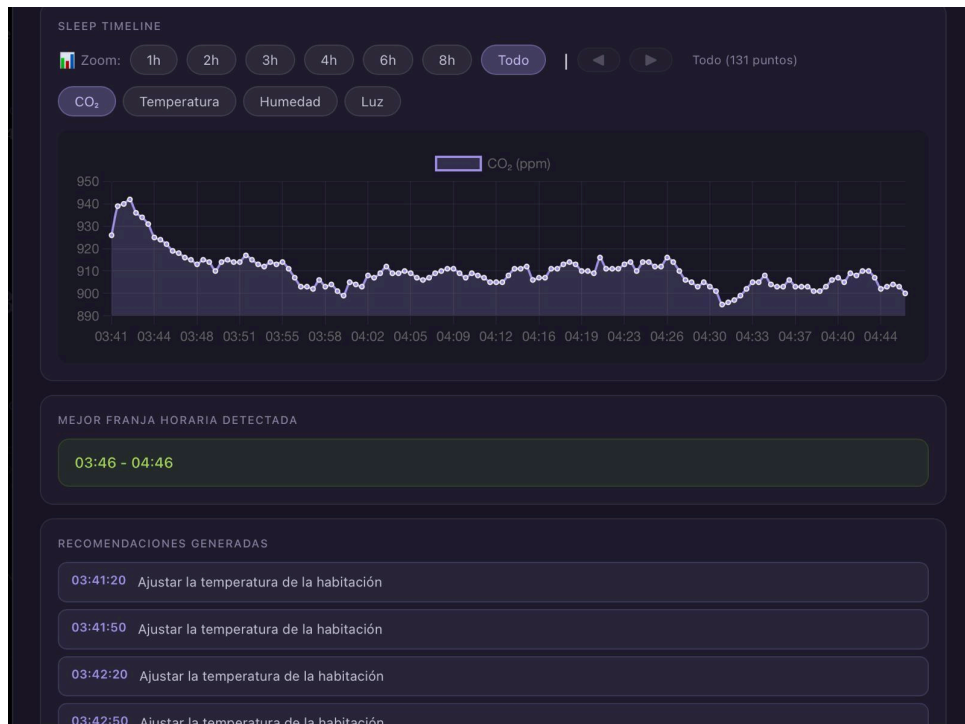
En definitiva, consideramos que el proyecto ha cumplido con todos los objetivos planteados inicialmente y ha dado como resultado un producto funcional, robusto y útil que puede ayudar a las personas a mejorar su descanso y, por tanto, su calidad de vida.

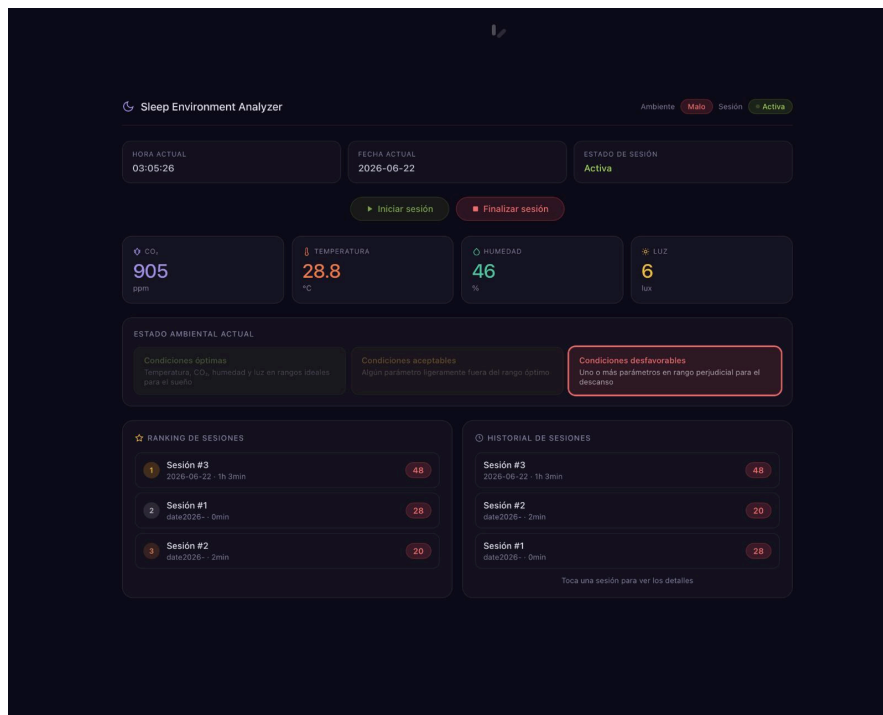
15. ANEXOS

15.1. Fotografías del hardware



15.2. Capturas de la interfaz web





15.3. Video demostrativo



<https://drive.google.com/file/d/1q095Y3H2u8qjhajUMhVHIJdmEOaKpRiC/view?usp=d>

[rive link](#)